



UNIVERSITY OF RWANDA

RMA- GAKO

COLLEGE OF SCIENCE AND
TECHNOLOGY

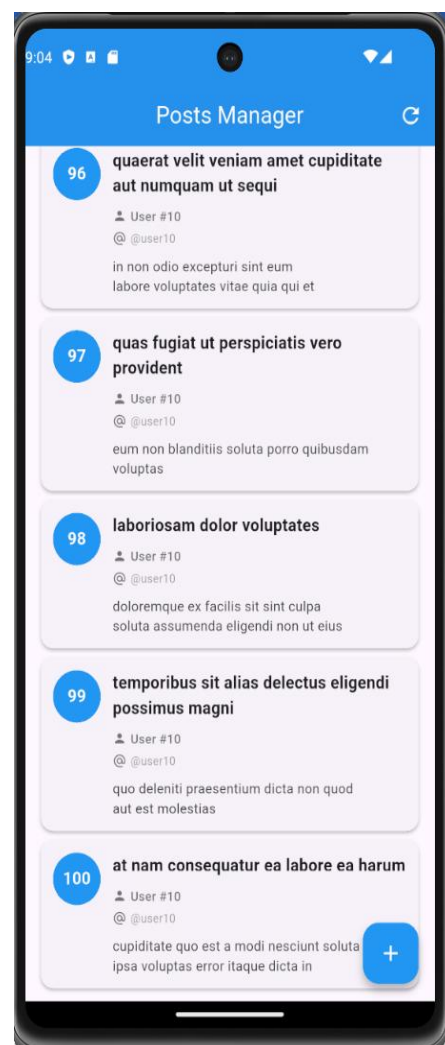
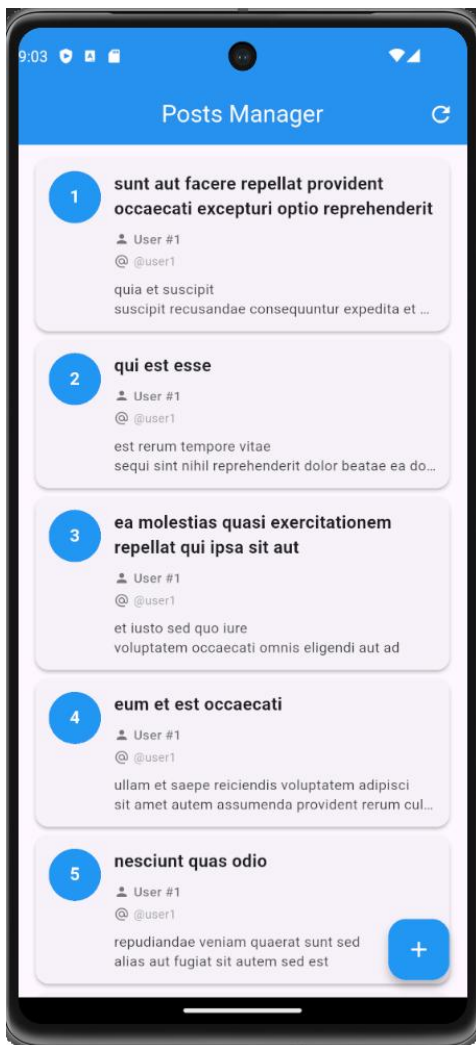
SCHOOL OF ICT

COMPUTERENGINEERING LEVEL
THREE

NAME: EDUARD GASASIRA
223026683

LAB 4: CONSUMING API

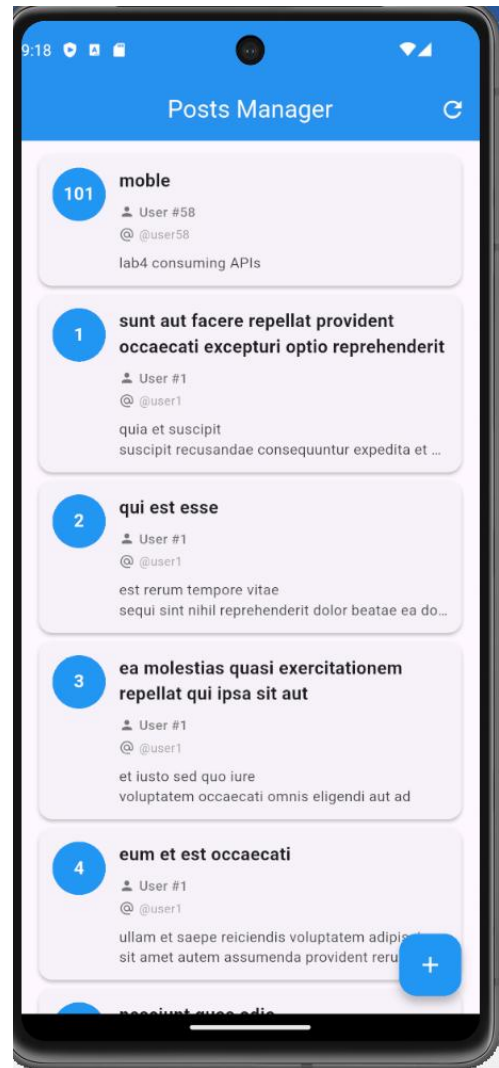
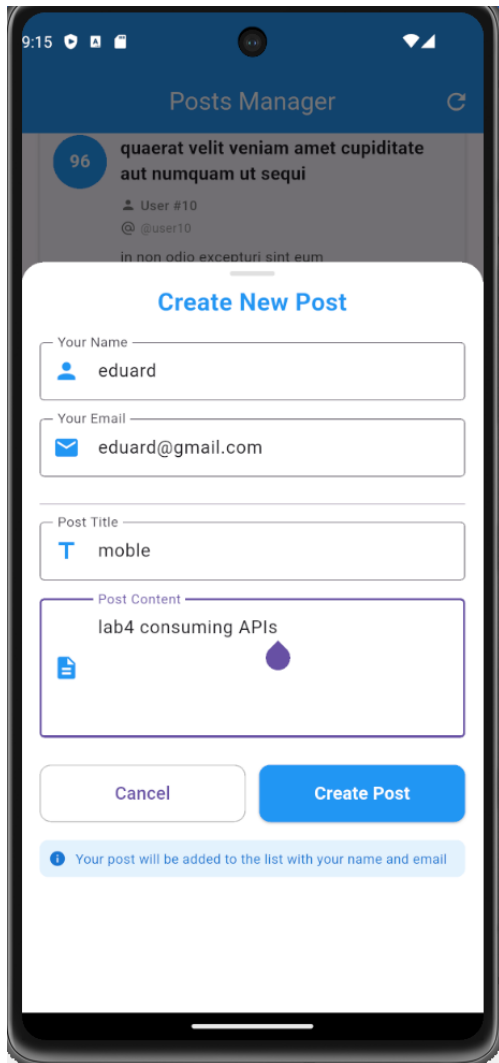
1. View all available posts



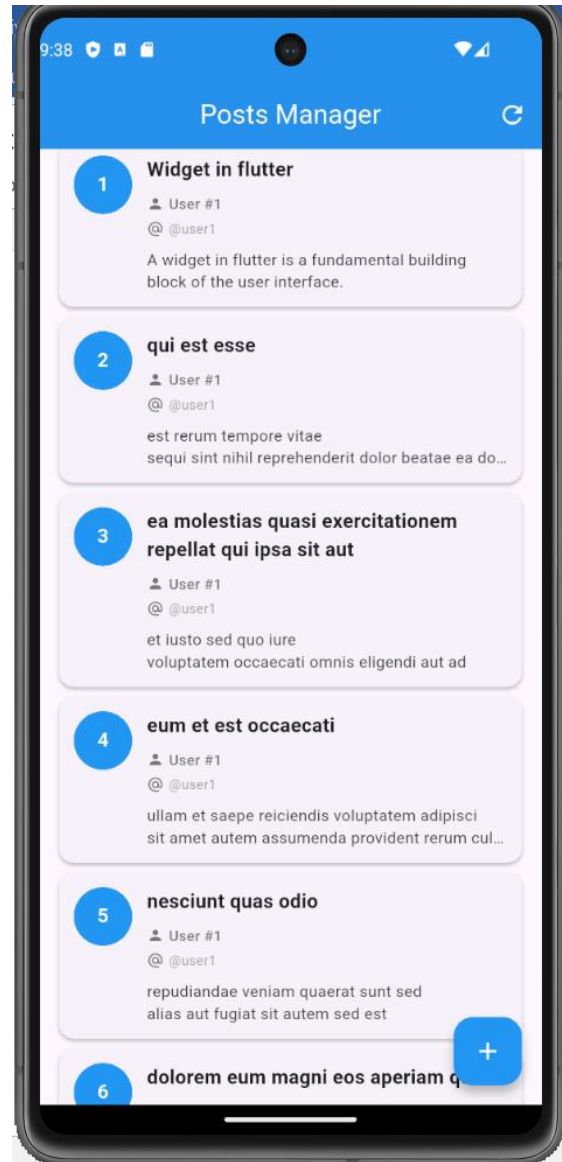
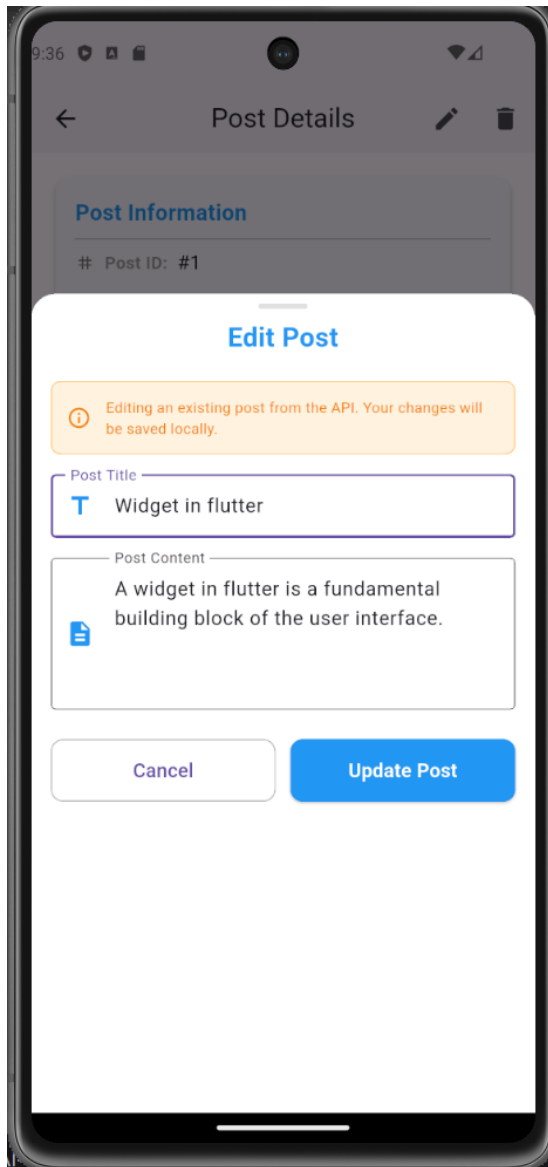
2. Read the details of a post



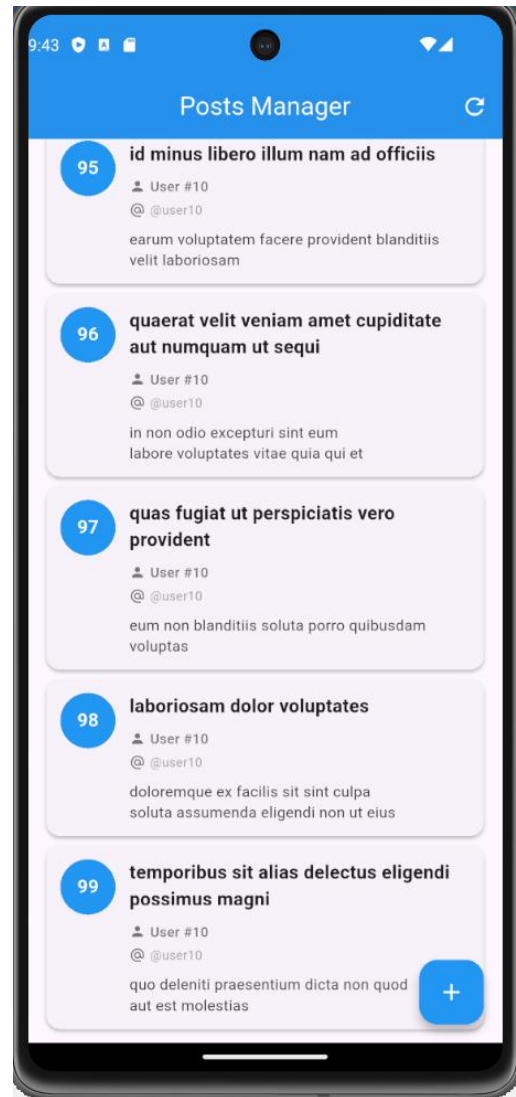
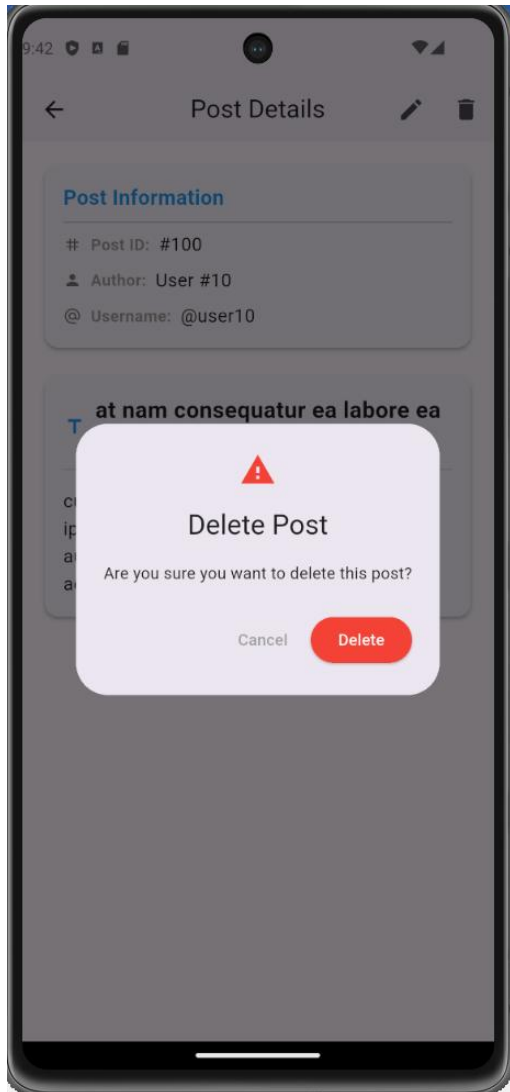
3. Create a new post



4. Edit an existing post



5. Delete a post



scanned handwritten materials:

①. Dependencies used and why

Core Dependencies

Dependency	Version	Purpose	Why used
Flutter	Sdk:flutter	Flutter SDK	Required for building Flutter applications
http	^1.1.0	HTTP client	Used to make API requests (GET, POST, PUT, DELETE) to JSONPlaceholder. It provides a simple, Future-based API for network operations
Provider	^6.1.1	State management	Manages application state across widgets. Provides efficient rebuilding only when data changes, preventing unnecessary widget rebuildings.

Why Provider over other state management solutions?

- ✓ Simplicity: Minimal boilerplate code compared to Bloc or Redux.
- ✓ Efficiency: Only rebuilds widgets that depend on changed data.
- ✓ Testability: Easy to mock providers for unit testing.

- ✓ Scoped access : provides data where needed without passing down through constructors.
- ✓ Flutter - native : Works seamlessly with Flutter's reactive programming model.

Development Dependencies

Dependency	Purpose
Flutter_test	for writing and running unit and widget tests.
Flutter_lints	Enforces Dart best practices and coding standards

② . Handling API Exceptions (Network Errors, Invalid Data)

Exception Handling Strategy

// In ApiService class

```
static Future <List <post >> fetchPosts() async {  
  try {
```

```
    final response = await http.get(Uri.parse(postsEndpoint));  
    if (response.statusCode == 200) {
```

```
      // success - parse data
```

```
      List <dynamic> jsonList = json.decode(response.body);  
      return jsonList.map(json) => post.fromJson(json)).toList();  
    } else {
```

```
      // server returned error status code
```

```
      throw Exception("failed to load posts: ${response.statusCode}");  
    }
```

```
  } catch (e) {
```

```
    // Network error, timeout, or other exceptions
```

```
    throw Exception('failed to connect to server: $e');
```

```
  }
```

```
}
```

Types of Exceptions Handled

Exception Type	Example	Handling Strategy
Network Errors	No internet connection, DNS failure	catch block displays "failed to connect to server" message with retry button.
HTTP Status Errors	404 Not found, 500 Server Error	Check status code, throw meaningful exception with status code.
JSON parsing Errors	Malformed response data	Try-catch around <code>json.decode()</code> , handle parsing exceptions.
Timeout Errors	Request takes too long.	http package handles internally, caught in catch block.
Invalid data	Missing required fields	Model validation in <code>fromJson()</code> with null safety.

UI-Level Error Handling

```
// In PostProvider
Future<void> fetchPosts() async {
  -isLoading = true;
  -error = null;
  notifyListeners();
  try {
    final results = await Future.wait([
      ApiService.fetchPosts(),
      ApiService.fetchUsers(),
    ]);
    -post = results[0];
    -user = results[1];
  } catch (e) {
    -error = e.toString();
  } finally {
    -isLoading = false;
    notifyListeners();
  }
}
```

Error Display in UI

```
if (provider.error != null) {
  return Center(
    child: Column(
      children: [
        Icon(Icons.outline, size: 64,
          color: Colors.red[300]),
        Text('Error loading posts'),
        Text(provider.error!),
        ElevatedButton(
          onPressed: () => provider.fetchPosts(),
          child: Text('Try Again'),
        ),
      ],
    ),
  );
}
```

3. Explain the future Widget have you used and the part of the UI Depends on the future.

What is FutureBuilder Widget?

In this app, I used FutureBuilder as the primary future Widget. FutureBuilder is a Flutter widget that builds itself based on the latest snapshot of interaction with a future.

Where FutureBuilder is used

In this app, FutureBuilder is used in the post-form widget to handle user data loading:

// In post-form, dark - user selection Dropdown

```
Consumer <PostProvider>(  
  builder: (context, provider, child) {  
    if (!isLoadingUsers || provider.users.isEmpty) {  
      return const SizedBox(  
        height: 70,  
        child: Center(  
          child: CircularProgressIndicator(),  
        ),  
      );  
    }  
    return DropdownButtonFormField<int>(  
      // Dropdown with loaded user data  
      items: provider.users.map((user) {  
        return DropdownMenuItem<int>(  
          value: user.id,  
          child: Column(  
            children: [  
              Text(user.name),  
              Text('@{user.username}'),  
            ],  
            mainAxisAlignment: MainAxisAlignment.start,  
          ),  
        );  
      }).toList(),  
    );  
  },  
);
```

01 Parts That Depends on Future

1. Posts List Screen - Loading State.

// In posts - List - screen . dart

```
if (provider.isLoading && provider.posts.isEmpty) {  
  return const center(  
    child: CircularProgressIndicator(),  
  );  
}
```

2. Posts List Screen - Error State

```
if (provider.error != null) {  
  return center(  
    child: column(  
      children: [  
        Icon(Icons.error_outline),  
        Text('Error loading posts'),  
        Text(provider.error!),  
        ElevatedButton(  
          onPressed: () => provider.fetchPosts(),  
          child: Text('Try Again'),  
        ),  
      ],  
    ),  
  );  
}
```

3. Post List Screen - Success State

```
if (Provider.posts.isEmpty) {  
  return const Center(child: Text('No posts available'));  
}  
return ListView.builder(  
  itemCount: Provider.posts.length,  
  itemBuilder: (ctx, index) {  
    return postCard(post: Provider.posts[index]);  
  }  
);
```

4. Post form - user loading state

```
if (!isLoadingUsers || Provider.users.isEmpty) {  
  return const SizedBox(  
    height: 70,  
    child: Center(  
      child: CircularProgressIndicator(),  
    ),  
  );  
}
```

